

Energy-Aware Arm and Gantry Selection in Ceiling Robots with Multi-Arm Configurations Using GNG

^{1st} [TODO: Given Name Surname]
[TODO: dept. name of organization]
[TODO: name of organization]
[TODO: City, Country]
[TODO: email address or ORCID]

Abstract—The two ceiling-mounted rail carrying four arms turns every pick into two coupled decisions: which arm should act, and where the rail should place it, since repositioning the rail is one of the dominant costs of the motion. We propose an energy-aware selection cost J that chooses the arm and the full eight-degree-of-freedom configuration together over a base-aware capability map built with Growing Neural Gas (GNG). Every map node stores a representative configuration together with its manipulability and the effort needed to hold it against gravity, so a node is both a cell of reachable space and a ready inverse-kinematics seed. Treating the rail as a degree of freedom when building the map, rather than sampling the arm alone, enlarges the measured reachable workspace by about five times, and a boundary-seeding step keeps the map accurate to within a few centimetres of the true reach surface. In simulation on NVIDIA Isaac Sim, GNG-seeded inverse kinematics and energy-aware selection are evaluated against nearest-, fixed-, and random-baseline policies for arm and base choice [TODO: summarize E2/E3 headline numbers once collected].

Index Terms—mobile manipulation, capability map, growing neural gas, redundant manipulator, base placement, energy-aware planning

I. INTRODUCTION

As global demographic shifts persist, particularly the accelerated growth of the aging population, there is an increasing demand for assistive technologies that can be utilized in the comfort of one's own residence [21], [22]. One such technology is an assistive robot that traverses a residence, with an increasing number of tasks assigned to it, including the retrieval of objects, the maintenance of order, and the delivery of items to the occupants [20], [23]. The utilization of robots within residential environments presents a more substantial set of challenges when compared to their application in Factory facilities or outdoor settings [25]. Domestic residences generally possess smaller, more cluttered spaces, and robots must also share these spaces with the residents at all times. In the case of mobile manipulator robots that traverse the floor, this can impede the mobility of residents, who must exercise caution when walking or performing activities within the residence to avoid collisions with the robot [26]. One potential solution to this problem is to elevate the robot entirely off the floor [27]. In the event that the robotic arm is suspended from the ceiling, the floor area will be considered safe for the residents. Consequently, the robot will be able to continue performing its duties as an in-home assistant from above without

interfering with the residents' activities [24]. The present study focuses on the analysis of a ceiling robot equipped with a dual gantry system. Each gantry is equipped with two six-degree-of-freedom (6 DOF) arms, thereby constituting a quad-arm cooperative overhead manipulator.

Each gantry is characterized by two distinct types of motion: linear and rotational. Consequently, each arm possesses eight degrees of freedom. A given object can typically be accessed by either arm, and from a multitude of gantry positions. The robot, therefore, confronts two coupled inquiries simultaneously: which gantry and arm should execute the task, and where should the gantry be positioned prior to its arrival? It is important to note that both inquiries are associated with certain costs. Repositioning the heavy base is a significant expense in mobile manipulation, both in terms of time and energy [5], [6], [9]. The implementation of a straightforward policy of always reaching with the nearest arm or leaving the gantry in its current position may not be the optimal decision. This policy has the potential to result in a substantial waste of gantry movement. The objective of this study was to identify a solution to the problem of selecting the arm and gantry placement that would result in a low energy output.

Reachability and capability maps are a common means of depicting the reachable regions and the mobility capabilities of an arm. These maps are extensively employed in determining the optimal positioning of a robot [1], [4]. Two assumptions constrain the analysis within the confines of our specific setting. Firstly, the majority of maps designed for arm robots are constructed with a fixed base. Consequently, they fail to account for the novel reachable space that emerges when the base itself is mobile. Secondly, the records are typically limited to indicating whether a cell is reachable, without incorporating the cost of accessing it or a configuration from which inverse kinematics can initiate [1]. Recent methodologies have been developed that augment operators' situational awareness with respect to obstacles and proximate objects [5], [7]. Nevertheless, these approaches continue to prioritize the determination of the optimal placement of a single arm over the selection of the most suitable arm(s) to utilize within the confines of an energy budget. Redundancy-resolution work has been shown to optimize a single arm's posture for measures such as manipulability [2]. However, it does not select between arms or consider the costly base degree of freedom. A crucial

element is absent from the current discourse: a representation that directly addresses the question of which arm and which base or gantry placement result in the least energy expenditure on a ceiling-rail robot.

The solution to the problem is presented by an energy cost function, J , which operates on a gantry-aware capability map that has been constructed with the aid of Growing Neural Gas (GNG) [3]. Each node in the map maintains a representative 8-DOF configuration, in conjunction with its manipulability and the effort required to sustain that posture against gravity [2]. A node, therefore, is characterized by dual properties: it is both a cell of reachable space and a prepared starting point for inverse kinematics. The present paper has three contributions. Firstly, an energy-aware selection cost J is introduced. This cost selects the arm and the full 8-DOF configuration, including rail travel, in contrast to settling for the nearest reachable pose. Secondly, a base-aware GNG capability map is constructed, whereby the ceiling rail is regarded as a genuine degree of freedom. This map enhances plain reachability by incorporating a per-node IK seed, a manipulability value, and a holding cost. The process of boundary-seeding ensures the maintenance of map accuracy along the perimeter of the accessible workspace. Thirdly, an evaluation of the comprehensive approach in Isaac Sim is conducted, encompassing the characterization of the map, the benchmarking of GNG-seeded IK, and the comparison of energy-aware selection against nearest, fixed, and random baselines. Open-vocabulary perception with YOLOE provides the perception front end, supplying the target objects that the selection pipeline then acts on [10].

The simulation results support each claim. Letting the rail act as a degree of freedom enlarges the reachable workspace by about five times compared with an arm-only baseline. GNG seeding raises IK success and lowers solve time against [TODO: none/KDL/voxel baselines – E2], and energy-aware selection cuts base travel and mechanical energy relative to the nearest, fixed, and random baselines by [TODO: E3 headline numbers]. All of these numbers come from Isaac Sim, and validation on the physical robot is left for future work. The rest of the paper is organized as follows. Section II reviews related work, Section III describes the system, and Section IV presents the base-aware GNG capability map. Section V defines the energy cost J and the selection procedure, Section VI reports the experiments, and Section VII concludes.

II. RELATED WORK

Reachability and capability maps. A long line of work represents where a manipulator can reach by discretizing its workspace and recording, for each cell, whether the end-effector can be placed there. Zacharias et al. turned this idea into a capability map that also captures how the arm can be oriented at each location, giving a richer picture than a plain binary occupancy grid [1]. Reuleaux extended the same principle to base placement by inverting the reachability map, so the robot can ask where it should stand to serve a target [4]. More recent representations focus on making

these maps cheaper to build and query. RM4D, for example, reduces the discretization from six dimensions to four and serves both reachability and inverse-reachability queries from one structure [8]. These maps are built for a single arm on a fixed base, and a cell usually stores reachability alone. Our map differs on both counts. It treats the ceiling rail as part of the configuration, so it records how base motion extends reach, and every node carries a ready inverse-kinematics seed together with a manipulability and a holding cost rather than a single occupancy bit.

Base placement for mobile manipulators. Deciding where a mobile base should stop before manipulating is a well-studied problem. Sandakalum et al. learn an inverse-reachability network that places the platform for a given task while accounting for obstacles [7]. MoMa-Pos optimizes base placement by reasoning about the kinematics of the objects to be manipulated [5], and BaSeNet plans a whole sequence of base poses so that a robot can pick up several objects in turn [6]. All of these methods answer where a single arm should be positioned. None of them chooses among several arms, and they do not rank candidate placements by the energy that reaching from them would cost. Our work adds both of these missing pieces for a redundant ceiling-rail robot.

Redundancy resolution and inverse kinematics. When a chain has more joints than the task requires, its extra freedom can be used to improve a secondary objective. Yoshikawa’s manipulability measure is a classic choice, steering a redundant arm away from singular postures [2]. Solving the inverse kinematics itself has also seen renewed interest through learning, with recent solvers such as IKDiffuser generating configurations from a diffusion model [17]. These approaches resolve the posture of one arm for one criterion at a time. They do not decide which of two arms should act, and they carry no notion of the base travel that a given posture would require. We keep the inverse kinematics solver itself lightweight and classical, and instead use the capability map to supply a good starting configuration, while the arm choice and the base motion are settled by the energy cost.

Dual-arm coordination. Systems with two arms raise the question of how to divide the task between them. Recent planners such as DAG-Plan and RoboPARA build dependency graphs over a task and assign subtasks to each arm so that the two can act in parallel [18], [19]. Their concern is the temporal and logical structure of a multi-step task. Ours is different and complementary. For a single reaching action we ask which arm reaches the target for the least energy, given that both arms share one moving base, and we answer it with the same map and cost that also place the base.

III. SYSTEM

A. Robot Platform and Kinematic Model

The robot consists of a single rail mounted to the ceiling that carries two arms side by side. Each arm is a Kinova Gen3 Lite, a six-joint manipulator with a reach of about 0.76 m and a two-finger gripper at its wrist [16]. The rail adds two more degrees of freedom that both arms share, a prismatic axis that

TODO figure asset — two-panel teaser: rail+arm frames/DOF (left) and the perception→selection→planning pipeline (right)

Fig. 1. Overview of the ceiling-rail dual-arm robot and the proposed pipeline. Left: a rail mounted to the ceiling carries two Kinova Gen3 Lite arms and adds a prismatic axis (d) and a revolute axis (θ) that both arms share, so each arm forms an eight-degree-of-freedom chain $\mathbf{q} = [d, \theta, q_1 \dots q_6]$; two overhead RGB-D cameras observe the workspace. Right: the end-to-end pipeline, from open-vocabulary perception, through energy-aware arm-and-base selection over the base-aware capability map, to collision-free planning and execution.

slides the carriage horizontally and a revolute axis that rotates it, so relative to the ground each arm forms an eight-degree-of-freedom chain. We write a configuration of one arm as $\mathbf{q} = [d, \theta, q_1, \dots, q_6]$, where d is the rail translation, θ the rail rotation, and $q_1 \dots q_6$ the arm joints. The kinematic tree is anchored at a fixed world frame. The rail base is attached to the world, each arm base is attached to the rail, and a tool frame at the gripper defines the end-effector pose that the task is specified in. Because the two arms are mounted on the same carriage, moving the rail moves both of them together, which is what couples the choice of arm to the placement of the base. Fig. 1 shows the platform and its frames alongside an overview of the end-to-end pipeline, from perception through energy-aware selection to planning and execution.

B. Simulation Environment

All experiments in this paper are carried out in NVIDIA Isaac Sim, a GPU-accelerated simulator with a physics engine suitable for contact-rich manipulation [11], [12]. The simulated robot is built from the same URDF description that defines the physical platform, so the kinematics and joint limits used in simulation match the real hardware. The scene places a set of everyday tabletop objects within the workspace, some within easy reach of both arms and at least one deliberately positioned near the edge of the reachable workspace (Fig. 2). The scene is observed by two RGB-D cameras placed above the workspace, which are the system’s only exteroceptive sensor. Their images serve two roles at once. The color stream feeds the object detector, and the depth stream builds the collision octomap used for planning, so a single sensing rig supports both perception and collision avoidance. We treat the simulator as the evaluation platform for this work and validate on the physical robot in future work.

C. Perception Front End

Target objects are supplied to the pipeline by an open-vocabulary perception module based on YOLOE, a real-time

TODO figure asset — Isaac Sim scene screenshot, overhead RGB-D view

Fig. 2. The simulated workspace in NVIDIA Isaac Sim, viewed from one of the two overhead RGB-D cameras, showing the tabletop objects used in the experiments; at least one object is placed near the edge of the reachable workspace.

model that detects and segments objects from free-form text prompts [10]. Running on the color image from each of the two overhead cameras, it returns a segmentation mask per object, which we back-project with the aligned depth and combine across the two viewpoints to estimate the object’s position in the world frame. This lets the operator ask for an object by name and have the system localize it without a fixed object database (Fig. 3). Perception acts as the enabling front end here, providing the target that the capability map and the energy cost then act on.

TODO figure asset — YOLOE mask+label overlay and back-projected object point

Fig. 3. Open-vocabulary perception. YOLOE detects and segments the requested object from a text prompt (masks and labels shown); the mask is back-projected with depth to localize the object in the world frame.

D. Planning and Execution Stack

Given a target and a candidate configuration, the motion layer is built on MoveIt 2 [13] with planners from the Open Motion Planning Library [14]. Inverse kinematics requests are answered by MoveIt against the current robot state, and collisions are checked against an octomap that is populated from the two cameras’ depth images, so a plan avoids both the scene and the robot’s own body. The object chosen as the target is excluded from the cloud that feeds the octomap so that it can be grasped, while the remaining objects and the environment stay as obstacles. The capability map of Section IV is built offline with the Pinocchio rigid-body-dynamics library [15], which we use for forward kinematics when sampling configurations and for the gravity torque that gives each map node its holding cost. At run time a candidate drawn from the map seeds the inverse-kinematics query, MoveIt plans a collision-free motion to it, and the motion is optionally

executed in simulation. The selection procedure that decides which candidate to try, and in which order, is the subject of Section V.

IV. BASE-AWARE GNG CAPABILITY MAP

The core of our method is a capability map of a single arm. For every location the end-effector can reach, the map stores a configuration that reaches it and a measure of how good that configuration is. Unlike a reachability map, which encodes only whether a location is attainable, this representation associates each reachable location with a concrete configuration and a quality measure, and it is base-aware in that the stored configuration incorporates the rail's degrees of freedom. We build one such capability map per arm, so the two arms sharing the rail are described by two maps that we query independently in Section V.

A. Sampling Reachable Configurations

We sample the eight-degree-of-freedom configuration $\mathbf{q} = [d, \theta, q_1, \dots, q_6]$ uniformly within the joint limits and compute the resulting end-effector pose by forward kinematics with the Pinocchio library [15]. For each sample we also record the manipulability $w = \sqrt{\det(JJ^T)}$ [2] and the gravity torque required to hold the configuration. This produces a dataset of (pose, $\mathbf{q}$, manipulability, holding cost) tuples that covers the combined arm-and-rail workspace. Because the rail translation d and rotation θ are sampled alongside the arm joints, the dataset already reflects the reach that base motion provides, not just the reach of a fixed arm.

B. Growing the Map

We fit a Growing Neural Gas network [3] to this dataset. Each node stores a vector $[\mathbf{x} \mid \mathbf{q}]$ that concatenates the end-effector position $\mathbf{x}$, which we call the task part, with the configuration $\mathbf{q}$ that produces it. Training is arranged so that the best-matching-unit search uses the task part alone, which makes the nodes tile the reachable workspace, while adaptation updates the whole vector, so each node's $\mathbf{q}$ converges to a configuration that is representative of its workspace cell. Edges are formed between co-activated nodes by competitive Hebbian learning, giving the map a topology over the workspace. Every node is therefore two things at once, a location the end-effector can reach and a ready inverse-kinematics seed for reaching it. This is the sense in which the map is base-aware. Because $\mathbf{q}$ carries the rail translation and rotation, two ways of reaching the same point from different base placements settle into different nodes, so a node's seed stays a valid single configuration rather than an average of incompatible base positions. To keep the map faithful at the reachable boundary, where nodes grown only from interior samples would otherwise settle inward, we pin a shell of nodes on the measured reach surface before growing the interior (Fig. 4), and Section VI reports the resulting edge fidelity.

TODO figure asset — RViz node cloud colored by manipulability, boundary shell visible — `view_gng.launch.py`
`model_path:=/tmp/arm1_model.npz`

Fig. 4. The base-aware GNG capability map for one arm, rendered in RViz. Nodes are colored by manipulability (blue near the workspace edge, red where the arm is dexterous); the pinned boundary shell reaches the true rail-extended reach surface.

C. Capability Layers

Beyond plain reachability, each node carries two quality values that the energy cost of Section V draws on. The manipulability w [2] records how dexterous the arm is at that node, low near the workspace edge and near singular postures and high where the arm can move freely in any direction. This quality layer is what separates our map from a binary reachability grid that only marks a cell reachable or not [1]. The holding cost, the norm of the gravity torque at the node's configuration, records how much effort it takes to hold that posture against gravity. Both values are computed once, offline, with the same Pinocchio model used for sampling [15] and stored on the node, so they are available at query time at no extra cost.

V. ENERGY-AWARE ARM AND BASE SELECTION

Given the object pose from perception and the two capability maps, the selection stage decides which arm to use and which eight-degree-of-freedom configuration to reach with. It makes both decisions together, by an energy cost evaluated over candidates drawn from the maps.

A. The Energy Cost

We score a candidate configuration by a weighted sum of the effort of using it,

$$J = w_{gl} \left(\frac{\Delta_{lin}}{\rho_{gl}} \right) + w_{gr} \left(\frac{\Delta_{rot}}{\rho_{gr}} \right) + w_{arm} \left(\frac{\Delta_{arm}}{\rho_{arm}} \right) + w_{dist} \left(\frac{e}{\rho_{dist}} \right) + w_{hold} \left(\frac{h}{\rho_{hold}} \right) - w_{manip} \left(\frac{m}{\rho_{manip}} \right) \quad (1)$$

where Δ_{lin} , Δ_{rot} , and Δ_{arm} are the rail-linear, rail-rotation, and summed arm-joint travel from the current state to the candidate, e is the distance from the arm's current tool frame to the object, h is the gravity torque needed to hold the candidate posture, and m is its manipulability [2]. The rail's linear and rotational axes carry separate weights because their units and their cost differ, metres of heavy-carriage travel against radians of rotation. Manipulability enters with a minus sign, so a more dexterous configuration is cheaper. Each term is divided by a fixed reference ρ before its weight is applied, which makes the terms dimensionless and of comparable magnitude so that a

weight reads directly as the priority of that term. We set each reference to the median value the term takes over a calibration set of picks, so that no term dominates the sum merely because of the units it is measured in, and we tune the weights on the same set rather than by hand. Table I lists the calibrated weights and references.

TABLE I
CALIBRATED WEIGHTS AND NORMALIZATION REFERENCES FOR THE ENERGY COST J . EACH REFERENCE ρ IS THE MEDIAN RAW VALUE OF ITS TERM OVER THE CALIBRATION SET OF PICKS.

Term	Symbol	Weight	Reference ρ
Rail linear travel	Δ_{lin}	2	0.95
Rail rotation travel	Δ_{rot}	12	0.70
Arm joint travel	Δ_{arm}	20	6.00
Tool-to-object distance	e	3	1.36
Gravity holding torque	h	1	2.90
Manipulability (subtracted)	m	1	0.145

The terms are proxies for the energy of carrying out the pick. Rail and arm travel are the mechanical work of moving each axis, with the heavy rail the dominant cost [5], [6]; the tool-to-object distance stands for the arm motion still needed to reach the object; and the holding term charges for fighting gravity. We do not claim that J is the exact energy of a pick, but Section VI shows that minimizing it lowers the measured mechanical energy of the executed trajectory relative to the baselines [9].

B. Pooling Candidates

For a target object, we gather from each arm’s capability map the nodes whose task position lies within a radius of the object, which serves as the reachability filter, and score each of them with J . The radius scales with the node spacing of the map, so the number of pooled candidates stays stable no matter how densely the map was grown. Because the two arms have separate maps, the pool holds candidates from both arms at once, each carrying its own base placement and full configuration, so scoring the pool compares the two arms and their base placements on the same scale.

C. Round-Robin Selection and Execution

We sort the pooled candidates by J and try them in order, but interleave the order across the two arms so that one arm cannot consume every attempt before the other is tried; the arm that holds the overall lowest- J candidate is given a small head start. For each candidate in turn we solve inverse kinematics, seeded by the candidate configuration, to a pre-grasp pose that stands off a fixed clearance above the top of the object with a top-down orientation, and we ask MoveIt for a collision-free plan. The search is plan-only, so the arm never moves during selection, and the first candidate that yields a valid plan is selected (Fig. 5). When execution is enabled, only the selected candidate is planned and executed, and if execution is aborted by a change in the scene the search falls through to the next candidate. The arm that wins and the rail placement its configuration implies are exactly the arm-and-base decision that this paper sets out to make.

TODO figure asset — top-view diagram, object + both arms’ base travel, J picks the cheaper (not nearest) arm – values from a logged E3 pick

Fig. 5. Energy-aware selection in a representative pick (values from a logged trial). The cost J prefers the arm and base placement with lower total energy, which here is not the arm whose end-effector is nearest the object.

VI. EXPERIMENTS

A. Map Characterization (E0)

Table II summarizes the base-aware GNG capability map built for each arm with `build_maps.sh` (80,000 FK samples, 3000 nodes, 600 pinned boundary nodes, $\lambda = 60$, 2 training epochs).

TABLE II
GNG CAPABILITY MAP PROPERTIES, PER ARM.

Property	arm_1	arm_2
FK samples	80,000	80,000
Surface points detected	8578	8459
Total nodes	3000	3000
Pinned boundary nodes	600	600
Interior nodes	2400	2400
Edges	15,896	15,965
Median node spacing (m)	0.176	0.176
Mean node spacing (m)	0.196	0.197
q DOF per node	8 (2 rail + 6 arm)	8

B. Reachable-Workspace Gain from the Rail DOF (E1)

We compare the reachable volume with the rail locked at a fixed pose (6 arm DOF only) against the rail sampled over its full travel (rail 0–3.0 m, rotation $\pm 180^\circ$), matching FK sample *density* rather than raw sample count between the two conditions to avoid an under-sampling bias. Table III reports the reachable volume at three voxel resolutions.

TABLE III
REACHABLE VOLUME, RAIL LOCKED VS. RAIL ACTIVE (ARM_1).

Res. (m)	Locked (m ³)	Active (m ³)	Gain
0.08	2.127	12.289	5.78×
0.05	1.920	9.787	5.10×

Treating the rail as a degree of freedom expands the reachable workspace by about $5\times$ ($5.10\times$ at 0.05 m resolution, robust bracket $5.1\text{--}5.8\times$ over saturated resolutions). The locked bounding box measures approximately $1.51 \times 1.52 \times 1.49$ m; the active bounding box measures approximately $5.19 \times 2.31 \times 1.49$ m, reflecting the rail sweeping the x -axis.

1) *Boundary-Seeding Ablation (E1b)*: Table IV measures how far the GNG node hull’s outer extent falls short of the true FK-sampled reach surface, with and without the pinned boundary shell of Section IV-B, using extent metrics (max reach radius and per-axis bounding-box extent) rather than a centroid-radial metric, since the rail-swept workspace is not star-convex about its centroid.

TABLE IV
REACHABLE-EDGE SHORTFALL WITH AND WITHOUT BOUNDARY SEEDING (ARM_1).

Model	#Nodes	Max radius (m)	Shortfall (m)
True surface	—	2.572	—
Boundary = 600	3000	2.569	0.003
Boundary = 0	2668	2.314	0.258

Pinning a boundary shell before growing the interior cuts the reachable-edge shortfall from 0.26 m to under 1 cm at the maximum reach radius, with the largest recovery along the rail’s travel axis (x : 0.545 m $\rightarrow$ 0.029 m shortfall).

TODO figure asset — (a) bar chart: reachable volume, rail locked vs. active (Table III, data ready); (b) boxplot: rail travel / mechanical energy per selection mode (E3, pending)

Fig. 6. Results. (a) Reachable-workspace volume with the rail locked versus active, showing the gain from treating the base as a DOF. (b) Rail travel and mechanical energy per selection mode (energy-aware vs. nearest, fixed, random).

C. GNG-Seeded Inverse Kinematics (E2)

[TODO: Pending. Compares GNG-seeded IK against no-seed, KDL random-restart, and a lightweight voxel-map seed baseline on success rate, solve time, and solution manipulability (see docs/experiment_plan.md, Section E2).]

D. Energy-Aware Arm and Base Selection (E3)

[TODO: Pending – the paper’s star experiment. Compares selection_mode $\in$ {energy, nearest, fixed, random} on rail travel, arm travel, and measured mechanical energy over a grid of target poses (see docs/experiment_plan.md, Section E3).]

E. End-to-End Pick (E6)

[TODO: Pending. Full perception-to-execution pick success rate and failure-mode breakdown in Isaac Sim (see docs/experiment_plan.md, Section E6).]

F. Perception Accuracy (E4, E5)

[TODO: Pending. YOLOE detection/localization error against simulator ground truth (E4) and reachable-vs-unreachable classification accuracy (E5); see docs/experiment_plan.md.]

VII. CONCLUSION

We presented an energy-aware selection cost that chooses both the arm and the full 8-DOF configuration, rail travel included, on a ceiling-rail dual-arm robot, built on top of a base-aware GNG capability map that treats the rail as a first-class degree of freedom. Sampling and growing the map with the rail active, rather than holding it fixed, is what lets a single representation answer both which arm and where the base should be, and a boundary-seeding step keeps that map accurate to within centimetres of the true reach surface. The results measured so far show that the rail contributes the majority of the reachable-workspace gain over an arm-only baseline; the remaining comparisons against IK-seeding and arm/base selection baselines are ongoing (Section VI) and will complete the case for the energy cost J . All results in this paper come from Isaac Sim; validating the approach on the physical ceiling-rail platform, and studying how sim-to-real gap affects the calibrated weights of J , is left for future work.

ACKNOWLEDGMENT

[TODO: Funding / acknowledgment text, or delete this section.]

REFERENCES

- [1] F. Zacharias, C. Borst, and G. Hirzinger, “Capturing robot workspace structure: representing robot capabilities,” in *Proc. IEEE/RSJ Int. Conf. Intelligent Robots and Systems (IROS)*, 2007, pp. 3229–3236, doi: 10.1109/IROS.2007.4399105.
- [2] T. Yoshikawa, “Manipulability of robotic mechanisms,” *Int. J. Robotics Research*, vol. 4, no. 2, pp. 3–9, 1985.
- [3] B. Fritzke, “A growing neural gas network learns topologies,” in *Advances in Neural Information Processing Systems 7 (NIPS)*, MIT Press, 1995, pp. 625–632.
- [4] A. Makhal and A. K. Goins, “Reuleaux: robot base placement by reachability analysis,” in *Proc. IEEE Int. Conf. Robotic Computing (IRC)*, 2018, arXiv:1710.01328.
- [5] B. Shao, N. Cao, Y. Ding, X. Wang, F. Gu, and C. Chen, “MoMa-Pos: an efficient object-kinematic-aware base placement optimization framework for mobile manipulation,” arXiv:2403.19940, 2024.
- [6] L. Naik, S. Kalkan, S. L. Sørensen, M. B. Kjærgaard, and N. Krüger, “BaSeNet: a learning-based mobile manipulator base pose sequence planning for pickup tasks,” in *Proc. IEEE/RSJ Int. Conf. Intelligent Robots and Systems (IROS)*, 2024, arXiv:2406.08653.
- [7] T. Sandakalun, N. X. Yao, and M. H. Ang, “Inv-Reach Net: deciding mobile platform placement for a given task,” in *Proc. IEEE-RAS Int. Conf. Humanoid Robots (Humanoids)*, 2022.
- [8] M. Rudorfer, “RM4D: a combined reachability and inverse reachability map for common 6-/7-axis robot arms by dimensionality reduction to 4D,” arXiv:2410.06968, 2024.
- [9] A. Gupta, “Energy efficiency in robotics software: a systematic literature review (2020–2024),” arXiv:2508.12170, 2025.
- [10] A. Wang, L. Liu, H. Chen, Z. Lin, J. Han, and G. Ding, “YOLOE: real-time seeing anything,” in *Proc. IEEE/CVF Int. Conf. Computer Vision (ICCV)*, 2025, arXiv:2503.07465.
- [11] S. Gao, M. Pagnucco, T. Bednarz, and Y. Song, “NVIDIA Isaac Sim: enabling scalable, GPU-accelerated simulation for robotics,” arXiv:2606.03551, 2026.
- [12] M. Mittal, C. Yu, Q. Yu, J. Liu, N. Rudin, D. Hoeller, *et al.*, “Orbit: a unified simulation framework for interactive robot learning environments,” *IEEE Robotics and Automation Letters*, vol. 8, no. 6, 2023.
- [13] D. Coleman, I. Şucan, S. Chitta, and N. Correll, “Reducing the barrier to entry of complex robotic software: a MoveIt! case study,” arXiv:1404.3785, 2014.
- [14] I. A. Şucan, M. Moll, and L. E. Kavraki, “The open motion planning library,” *IEEE Robotics & Automation Magazine*, vol. 19, no. 4, pp. 72–82, 2012.

- [15] J. Carpentier, G. Saurel, G. Buondonno, J. Mirabel, F. Lamiriaux, O. Stasse, and N. Mansard, "The Pinocchio C++ library: a fast and flexible implementation of rigid body dynamics algorithms and their analytical derivatives," in *Proc. IEEE/SICE Int. Symp. System Integration (SII)*, 2019, pp. 614–619, doi: 10.1109/SII.2019.8700380.
- [16] Kinova Robotics, "Gen3 lite robot – user guide / product specifications," [Online]. Available: <https://www.kinovarobotics.com>
- [17] Z. Zhang and Z. Jiao, "IKDiffuser: a diffusion-based generative inverse kinematics solver for kinematic trees," arXiv:2506.13087, 2025.
- [18] S. Duan, P. Ren, N. Jiang, Z. Che, J. Tang, Z. Fan, Y. Sun, and W. Wu, "RoboPARA: dual-arm robot planning with parallel allocation and recomposition across tasks," arXiv:2506.06683, 2025.
- [19] Z. Gao, Y. Mu, J. Qu, M. Hu, S. Peng, C. Hou, L. Guo, P. Luo, S. Zhang, and Y. Lu, "DAG-Plan: generating directed acyclic dependency graphs for dual-arm cooperative planning," arXiv:2406.09953, 2024.
- [20] M. Lisondra, B. Benhabib, and G. Nejat, "Embodied AI with foundation models for mobile service robots: a systematic review," arXiv:2505.20503, 2025.
- [21] United Nations, Department of Economic and Social Affairs, Population Division, "World Population Prospects 2024: Summary of Results," United Nations, New York, NY, USA, 2024.
- [22] World Health Organization, "WHO Clinical Consortium on Healthy Ageing 2024: Meeting Report," World Health Organization, Geneva, Switzerland, 2025.
- [23] International Federation of Robotics, *World Robotics 2025 – Service Robots* (Executive Summary), 2025.
- [24] W. A. D. M. Jayatilaka *et al.*, "A novel hybrid robot configuration for enhanced accessibility and space efficiency," in *Proceedings in Technology Transfer*, Springer, 2025, doi: 10.1007/978-981-96-1399-1_22.
- [25] C. C. Kemp, A. Edsinger, and E. Torres-Jara, "Challenges for physical human-robot interaction," *IEEE Robotics & Automation Magazine*, vol. 14, no. 1, pp. 22–29, 2007, doi: 10.1109/MRA.2007.339622.
- [26] E. A. Sisbot, L. F. Marin-Urias, R. Alami, and T. Siméon, "A human-aware mobile robot motion planner," *IEEE Transactions on Robotics*, vol. 23, no. 5, pp. 874–883, 2007, doi: 10.1109/TRO.2007.904911.
- [27] H. Toyama, T. Takahashi, H. Ueno, *et al.*, "Development of a ceiling-mounted assistive robot system for home care," in *Proc. IEEE Int. Conf. Robotics and Automation (ICRA)*, 2002, vol. 4, pp. 3841–3846, doi: 10.1109/ROBOT.2002.1014387.